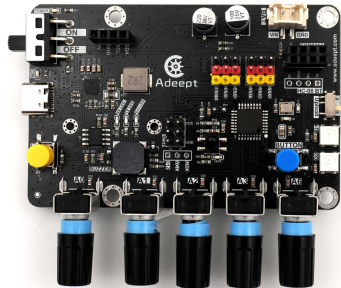


Lesson 7 Battery Level Detection

7.1 Overview

This course focuses on battery level detection, mainly introducing how to use Adeept Robotic Arm Control Board and Arduino development environment to monitor battery level. Through learning, readers will master the principle of battery voltage acquisition, hardware connection methods, and code writing methods.

7.2 Required Components

Components	Quantity	Picture
Adeept Robotic Arm Control Board	1	
Type-C USB Cable	1	
OLED Screen	1	

7.3 Principle Introduction

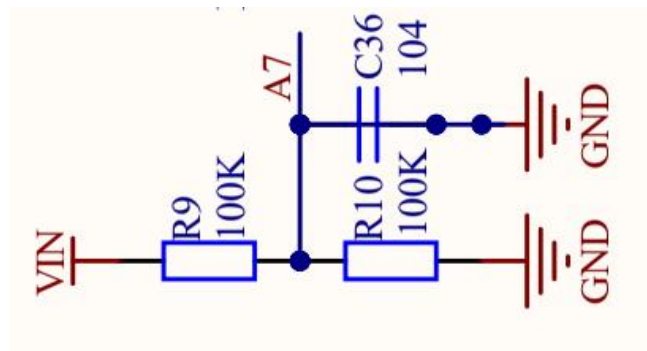
The voltage collection range of pin A7 on the Arduino Pixie board is 0-5V. However, the robot in this experiment is powered by two 18650 lithium batteries, which can reach a voltage of 8.4V when fully charged—exceeding this collection range. To address this issue, we have introduced two voltage divider resistors with the same resistance value, reducing the maximum voltage measured by pin A7 to 4.2V. Subsequently, the digital value is obtained via the Arduino's ADC (Analog-to-Digital Conversion) function, and the actual battery voltage can be calculated by combining this ADC value with the parameters of the voltage divider resistors for our observation and reference.

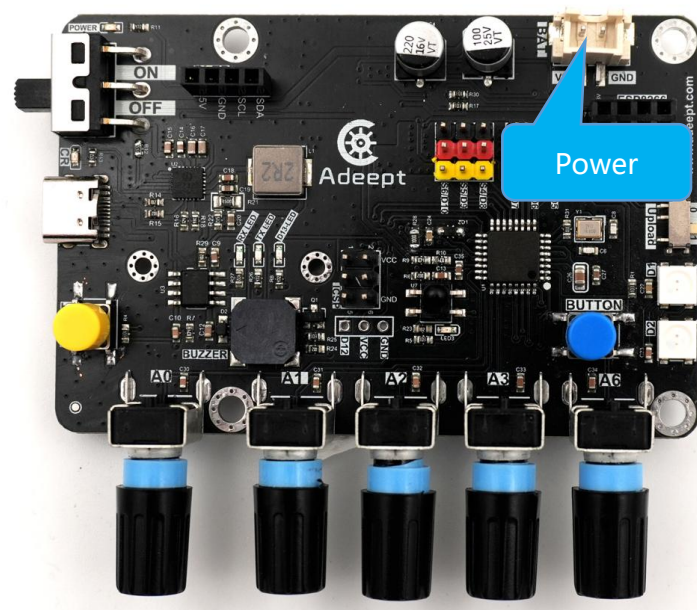
The Arduino ADC has a default precision of 10 bits, with a digital value range of 0-1023, corresponding to a reference voltage range of 0-5V. The voltage calculation formula is as follows:

$$\text{Actual Voltage (Vin)} = (\text{ADC Reading} \div 1023) \times \text{Reference Voltage (Vref)} \times \text{Voltage Division Ratio } ((R9+R10)/R9)$$

Our control board adopts a reference voltage of approximately 5.1 V. Minor errors may exist in the calculation results, which is normal.

7.4 Wiring Diagram





7.5 Demonstration

1. Connect your computer and Adeept Robotic Arm Control Board with a USB cable.
2. Open " 07_Battery_Level_Detection" folder in "/Code" ,double-click "07_Battery_Level_Detection.ino" .

```
07_Battery_Level_Detection | Arduino IDE 2.3.10
File Edit Sketch Tools Help
Arduino UNO
07_Battery_Level_Detection.ino
1  #include <Wire.h>
2  #include <SSD1306AsciiWire.h>
3
4  #define OLED_I2C_ADDR  0x3C
5  SSD1306AsciiWire display;
6
7  #define PIN_BATTERY      A7
8  const float ADC_REF_VOLTAGE  = 5.1;
9  const float VOLTAGE_DIV_RATIO  = 2.0;
10 const float BAT_MIN_VOLTAGE  = 6.0;
11 const float BAT_MAX_VOLTAGE  = 8.4;
12 float batteryVoltage = 0.0;
13 int lastBatADC  = 0;
14 float lastBatVolt  = 0.0;
15 int lastBatLevel  = 0;
16
17 int Get_Battery_Voltage_ADC(){
18     int adcValue = 0;
19     for (int i = 0; i < 5; i++){
20         adcValue += analogRead(PIN_BATTERY);
21         delayMicroseconds(200);
22     }
23     return adcValue / 5;
24 }
```

3. Select development board and serial port.

Board: **Tools---**>**Board---**>**Arduino AVR Boards---**>**Arduino Uno**

Port: **Tools --->Port---**>**COMx**

Note: The port number will be different in different computers.



4. After opening, click  to upload the code program to the Arduino. If there is no error warning in the console below, it means that the Upload is successful.

```
Output
Sketch uses 2952 bytes (9%) of program storage space. Maximum is 32256 bytes.
Global variables use 82 bytes (4%) of dynamic memory, leaving 1966 bytes for local variables. Maximum is 2048 bytes.
```

5. After successfully uploading the code, run the program. At this point, the OLED will display the ADC value of the battery and the calculated battery voltage value in real-time, measured in volts.

7.6 Code

```
01 #include <Wire.h>
02 #include <SSD1306AsciiWire.h>
03
04 #define OLED_I2C_ADDR 0x3C
05 SSD1306AsciiWire display;
06
07 #define PIN_BATTERY A7
08 const float ADC_REF_VOLTAGE = 5.1;
09 const float VOLTAGE_DIV_RATIO = 2.0;
10 const float BAT_MIN_VOLTAGE = 6.0;
11 const float BAT_MAX_VOLTAGE = 8.4;
12 float batteryVoltage = 0.0;
13 int lastBatADC = 0;
14 float lastBatVolt = 0.0;
15 int lastBatLevel = 0;
16
17 int Get_Battery_Voltage_ADC(){
18     int adcValue = 0;
19     for (int i = 0; i < 5; i++){
20         adcValue += analogRead(PIN_BATTERY);
21         delayMicroseconds(200);
22     }
23     return adcValue / 5;
24 }
25
```

```
26 float Get_Battery_Voltage(){
27     int adcValue = Get_Battery_Voltage_ADC();
28     batteryVoltage = ((float)adcValue / 1023.0) * ADC_REF_VOLTAGE * VOLTAGE_DIV_RATIO;
29     batteryVoltage = constrain(batteryVoltage, 0, BAT_MAX_VOLTAGE);
30     return batteryVoltage;
31 }
32
33 int Get_Battery_Ratio(){
34     float voltage = Get_Battery_Voltage();
35     int ratio = map(voltage * 100, BAT_MIN_VOLTAGE * 100, BAT_MAX_VOLTAGE * 100, 0, 100);
36     return constrain(ratio, 0, 100);
37 }
38
39 void DrawBatteryUI(int adc, float volt, int level){
40     display.clear();
41     display.set1X();
42
43     display.setCursor(10, 0);
44     display.print(F("Battery Monitor"));
45
46     display.setCursor(10, 3);
47     display.print(F("Bat ADC: "));
48     display.print(adc);
49
50     display.setCursor(10, 4);
51     display.print(F("Bat Voltage: "));
52     display.print(volt, 2);
53     display.print(F("V"));
54
55     display.setCursor(10, 5);
56     display.print(F("Bat Level: "));
57     display.print(level);
58     display.print(F("%"));
59 }
60
61 void setup(){
62     Serial.begin(9600);
63     pinMode(PIN_BATTERY, INPUT);
64     Wire.begin();
65     display.begin(&Adafruit128x64, OLED_I2C_ADDR);
66     display.setFont(Adafruit5x7);
67     display.clear();
68
69     int initADC = Get_Battery_Voltage_ADC();
70     float initVolt = Get_Battery_Voltage();
71     int initLevel = Get_Battery_Ratio();
72     DrawBatteryUI(initADC, initVolt, initLevel);
73
74     lastBatADC = initADC;
75     lastBatVolt = initVolt;
76     lastBatLevel = initLevel;
77 }
78
79 void loop(){
80     int curBatADC = Get_Battery_Voltage_ADC();
81     float curBatVolt = Get_Battery_Voltage();
```

```
82     int curBatLevel = Get_Battery_Ratio();
83     if ((curBatADC != lastBatADC) || (abs(curBatVolt - lastBatVolt) > 0.01f) || (curBatLevel !=
84     lastBatLevel)){
85         DrawBatteryUI(curBatADC, curBatVolt, curBatLevel);
86         lastBatADC = curBatADC;
87         lastBatVolt = curBatVolt;
88         lastBatLevel = curBatLevel;
89     }
90     delay(1000);
}
```